Dynamic programming solutions

A05-C01

Remaining length is insufficient. A still has one piece available and can sell the length-2 remainder whole. B has already used its allowance and cannot sell any positive remainder. Their feasible continuations differ.

Define $F[j, k]$ as the maximum future revenue using exactly length j and at most k pieces. Set $F[0, k] = 0$ for $k \geq 0$ and $F[j, 0] = -\infty$ for $j > 0$. For $j, k > 0$,

$$F[j, k] = \max_{1 \leq i \leq j} (p[i] + F[j - i, k - 1]).$$

Each candidate consumes one piece allowance. A corresponds to $F[2, 1] = 5$; B to $F[2, 0] = -\infty$. The original answer $F[4, 2] = 10$ uses two pieces of length 2. Negative infinity denotes infeasibility, not an ordinary negative revenue.

A05-C02

Candidates are listed in increasing i:

j	Candidates $p[i] + r[j - i]$	$r[j]$	first[j]
0	Empty	0	—
1	2	2	1
2	4, 5	5	2
3	7, 7, 7	7	1
4	9, 10, 9, 8	10	2

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

At length 3, all candidates tie at 7. Strict improvement keeps the first choice, $i = 1$. Reconstruction takes 2 from length 4 and then 2 from length 2, yielding $[2, 2]$. Its length is 4 and revenue is $5 + 5 = 10$. The tie rule fixes an output without changing the optimum value.

A05-C03

An actual optimum begins with some feasible i. Its remaining revenue is at most $r[j - i]$, so $OPT(j) \leq p[i] + r[j - i] \leq r[j]$.

Conversely, for every feasible i, concatenate a finished piece of length i with a decomposition attaining $r[j - i]$. This is feasible and uses exactly j. Hence each candidate is at most $OPT(j)$, giving $r[j] \leq OPT(j)$. The empty decomposition establishes the base; induction proves all entries.

Restricting to $i < j$ omits selling the entire rod uncut. For $p = [1, 5]$, the length-2 optimum is 5, while requiring a cut earns only 2. Length 1 would have no candidates at all.

Bound the true optimum from both directions

Optimal solution → its first piece i and remainder
OPT(j) ≤ p[i] + r[j−i] ≤ max candidates

Every candidate → a feasible full decomposition
p[i] + r[j−i] ≤ OPT(j) ⇒ max candidates ≤ OPT(j)

Therefore: r[j] = OPT(j)

A05-C04

The states are $0, 1, 2, 3, 4$: 5 in total. Each $j > 0$ depends on every $0 \le k < j$, giving $1 + 2 + 3 + 4 = 10$ edges.

Naive calls obey $T(0) = 1$ and $T(j) = 1 + \sum_{k=0}^{j-1} T(k)$. Thus $T(j) = 2T(j-1) = 2^j$ for $j \ge 1$, so $T(4) = 16$, including root and repeated base calls.

Memoization expands 4 nonbase states, whose loops examine 1, 2, 3, and 4 candidates: 10 evaluations. In general: $n + 1$ states, $n(n+1)/2$ edges, 2^n naive calls, n nonbase expansions, and $n(n+1)/2$ candidate evaluations. Cache hits still cost constant time; expanding n states does not imply linear total time.

A05-C05

The suffix $c \to d$ has length 1. Its independently longest replacement $c \to b \to a \to d$ has length 3. Substitution gives $a \to b \to c \to b \to a \to d$, repeating a and b. Vertices consumed by the prefix cannot be reused.

Fix start a. Let $D[S, v]$ optimize simple paths from a to v using exactly S. Set $D[\{a\}, a] = 0$ and all other infeasible states to $-\infty$. For $a, v \in S, v \ne a$,

$$D[S, v] = \max_{u \in S \setminus \{v\}, uv \in E} \{D[S \setminus \{v\}, u] + 1\}.$$

The previous set excludes the newly appended v, preventing repetition. Conversely, every nonempty feasible path supplies a last-edge candidate. Evaluate in increasing $|S|$ and take $\max_{S \ni a,d} D[S, d] = 3$. The bound $O(2^{|V|}|V|^2)$ is exponential, not necessarily polynomial. The failed approach was independent composition using a small state, not every possible DP.

A05-C06

(a) $r[0] = 0$, $r[1] = -2$, and $r[2] = \max(-2 - 2, -5) = -4$. Both choices during reconstruction have length 1, yielding $[1, 1]$. An initial maximum of 0 preserves an infeasible value, implicitly allowing all material to be discarded.

(b) Use $r[j] = \max_i(p[i] + r[j - i] - 2\mathbf{1}_{i<j})$. When $i < j$, separating the first piece makes one actual cut; $i = j$ makes none.

j	Candidates in increasing i	$r[j]$	First piece
0	Empty	0	—
1	1	1	1
2	0, 5	5	2
3	4, 4, 8	8	3
4	7, 8, 7, 9	9	4

Reconstruction follows $4 \to 0$, returning $[4]$ with revenue 9 and no cuts. Two pieces of length 2 would earn only 8 after one cut charge.

(c) Return revenue 0 and an empty list without accessing prices or the choice array. This same base makes the uncut candidate work.

A05-K01 — Complete table and subset

The complete table is:

$i \backslash c$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	10	10	10	10
2	0	6	10	16	16	16
3	0	6	10	16	18	22

Row 1 is zero below capacity 2 and 10 thereafter. Row 2 compares row 1 against $6 + D[1, c-1]$. Row 3, for capacities at least 3, compares row 2 against $12 + D[2, c-3]$. For example, $D[3,4] = \max(16, 12+6) = 18$. Trace $(3,5) \to (2,2) \to (1,2) \to (0,0)$: take 3, skip 2, take 1. The set is $\{1,3\}$, weight 5 and value 22.

A05-K02 — The missing item index

The recurrence gives $F(1) = 4, F(2) = 8$, although the only item contributes at most 4. Capacity alone forgets which items remain, allowing reuse. Define $D[i,c]$ for the first i items and an at-most capacity: $D[1,2] = \max(D[0,2], 4 + D[0,1]) = 4$. Taking item 1 leaves prefix 0. The proposed recurrence instead models an unlimited supply of this single type.

A05-K03 — Choose direction from the invariant

Ascending: after $c=2$, $[0,0,3,0,0]$; after $c=3$, $[0,0,3,3,0]$; after $c=4$, $[0,0,3,3,6]$. The last step reads this pass's new $dp[2]$, selecting two copies; this matches unbounded knapsack. Descending: after $c=4$, $[0,0,0,0,3]$; after $c=3$, $[0,0,0,3,3]$; after $c=2$, $[0,0,3,3,3]$. Each read uses a smaller, unmodified capacity, so the item is used at most once.

A05-K04 — Exact weight and negative values

All subsets have (weight,value): empty $(0,0)$, $\{1\}:(2,-3)$, $\{2\}:(3,4)$, and $\{1,2\}:(5,1)$. At most 5 chooses $\{2\}$ for 4; exactly 5 requires both for 1; exactly 1 is infeasible. Initial row: $[0,-\infty,-\infty,-\infty,-\infty,-\infty]$. Row 1: $[0,-\infty,-3,-\infty,-\infty,-\infty]$. Final row: $[0,-\infty,-3,4,-\infty,1]$. All zeros would invent states such as weight 1 with no items and can incorrectly replace a forced negative-value feasible solution with zero.

A05-K05 — Ties and reconstruction

The rows for capacities 0 and 1 are $[0,0]$, $[0,2]$, $[0,2]$. At $(2,1)$ equality skips item 2; at $(1,1)$ the improvement takes item 1, giving $\{1\}$. The set $\{2\}$ also has value 2, so the optimum is not unique. The final array $[0,2]$ lacks the prefix-specific values $D[i-1,c]$ required by traceback. Retain the table, store decisions with item-layer information, or recompute to trade time for space. A capacity pointer can be overwritten later and does not automatically preserve history.

A05-K06 — Greedy failure and a valid bound

Densities are item 2:6, item 1:5, item 3:4. Greedy takes 2 and 1, leaving capacity 1, so its value is 16. Feasible pairs are $\{1,2\}$ worth 16 and $\{2,3\}$ worth 18; $\{1,3\}$ and the triple are overweight. Thus the optimum is 18. Fractional selection takes 2, 1, and $1/3$ of 3, weight 4 and value 20; equal-weight exchanges establish its optimality. Every 0/1 completion is feasible for the relaxation, so the relaxed optimum upper-bounds the integer optimum. A node bounded by 17 cannot improve incumbent 18 and may be pruned. A bound of 20 leaves possible improvement and alone does not justify pruning.

A05-K07 — Input length and boundaries

Capacity itself needs only $b+1$ bits, but the table has 2^b+1 columns. For example, with a fixed number of items of comparable weight scale, input length is $O(b)$ while updates may be exponential in b. The algorithm is pseudopolynomial. If $W=0$ and all weights are positive, only the empty set fits, giving 0. With all-negative values and an allowed empty set, 0 is also optimal. Unlimited zero-weight positive-value copies yield arbitrarily large value at no weight cost; the objective is unbounded. Exclude that input or detect it explicitly rather than returning a finite DP value.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Sequence solutions

A05-S01

Rows index X; columns index Y.

$i:X_i \backslash j:Y_j$	0:∅	1:B	2:A	3:C	4:A
0:∅	0	0	0	0	0
1:A	0	0	1	1	1
2:B	0	1	1	1	1
3:A	0	1	2	2	2
4:C	0	1	2	3	3

For example, $C[3,2]=C[2,1]+1=2$ and $C[4,4]=\max(2,3)=3$. Move left from $(4,4)$, then take diagonal C, A, B matches, recording CAB backward. Reverse to obtain BAC, with indices $(2,3,4)$ in X and $(1,2,3)$ in Y. The final value bounds the optimum by 3, and this witness attains 3.

A05-S02

The assertion confuses an optimal string with all its index embeddings. The only optimal string is A, but either (x_1, y_1) or (x_1, y_2) embeds it. The first matching omits the last position of Y.

The correct recurrence remains $C[i, j] = C[i-1, j-1] + 1$ when both final symbols are a. Lower bound: append the final a to an LCS of the shorter prefixes. Upper bound: any optimal string must end in a, or another a could be appended. Removing its final a leaves a string that fits both shorter prefixes, of length at most $C[i-1, j-1]$. This works regardless of the original positions used for its last a. Here $C[1, 2] = C[0, 1] + 1 = 1$.

A05-S03

The complete route is:

Step	Current → predecessor	Record
1	$(7, 6) \to (6, 6)$	—; prefer above on tie
2	$(6, 6) \to (5, 5)$	A
3	$(5, 5) \to (4, 5)$	—
4	$(4, 5) \to (3, 4)$	B
5	$(3, 4) \to (3, 3)$	—
6	$(3, 3) \to (2, 2)$	C
7	$(2, 2) \to (2, 1)$	—
8	$(2, 1) \to (1, 0)$	B

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Reverse the recorded ABCB to obtain BCBA. Another optimum is BCAB, using X indices $(2, 3, 6, 7)$ and Y indices $(1, 3, 4, 5)$. A third is BDAB, using $(2, 5, 6, 7)$ and $(1, 2, 4, 5)$. All have the table's optimal length 4. Repeated symbols can also give multiple embeddings of the same output string.

A05-S04

Columns are ∅, B, A, A:

Completed prefix	Row values
∅	0, 0, 0, 0
A	0, 0, 1, 1
AB	0, 1, 1, 1
ABA	0, 1, 2, 2

The length is 2. For example, BA uses indices $(2, 3)$ in X and $(1, 2)$ in Y. Scan columns in increasing order because $cur[j]$ needs the completed $cur[j-1]$.

Only rows $i = 2, 3$ survive. On reaching row 2, traceback may need row 1, which has been overwritten. Remedies: retain a complete value or choice table from the beginning; or use a separate divide-and-conquer reconstruction/recompute subproblems with explicit time costs. The two-row code alone does not supply the history for ordinary $O(m+n)$ traceback.

A05-S05

Each boundary symbol costs one gap:

$i : X_i \backslash j : Y_j$	0:∅	1:A	2:G	3:C	4:A	5:C
0:∅	0	−3	−6	−9	−12	−15
1:A	−3	3	0	−3	−6	−9
2:C	−6	0	1	3	0	−3
3:G	−9	−3	3	0	1	−2
4:T	−12	−6	0	1	−2	−1
5:A	−15	−9	−3	−2	4	1

$F[5,5] = \max(F[4,4]-2, F[4,5]-3, F[5,4]-3) = \max(-4,-4,1) = 1$. Trace $(5,5), (5,4), (4,3), (3,2), (2,1), (1,1), (0,0)$ and reverse the columns:

```
ACGTA-
A-GCAC
```

The six scores are $3, -3, 3, -2, 3, -3$, totaling 1. A feasible score-1 alignment only proves the optimum is at least 1. The complete three-case recurrence and filled table prove that every alignment scores at most $F[5,5] = 1$. Both bounds establish optimality.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A05-S06

For the first case, correct boundaries give $F[1,0] = F[0,1] = -3$ and $F[1,1] = \max(-2,-6,-6) = -2$. Incorrect zero boundaries give $\max(-2,-3,-3) = -2$, accidentally the same result.

For A versus CA, the correct top row is $(0,-3,-6)$ and the first column is $(0,-3)$. Then $F[1,1] = -2$ and $F[1,2] = \max(-3+3,-6-3,-2-3) = 0$, witnessed by `-A / CA`.

With zero boundaries, $F[1,1] = -2$ but $F[1,2] = \max(0+3,0-3,-2-3) = 3$. The initial C is skipped for free before rewarding A/A. Passing one test does not establish correct boundary semantics; the second input exposes the free-prefix error.

A05-S07

The LCS has length 3, witnessed by BAC. The longest common substring also has length 3: BAC occurs at $X[2..4]$ and $Y[1..3]$. Equal answers do not identify the problems: ABC versus AC has LCS AC of length 2, but longest common substring length 1.

Define $H[i,j]$ as the longest common substring required to end at both x_i and y_j. Set $H[0,j] = H[i,0] = 0$. Equal symbols give $H[i,j] = H[i-1,j-1] + 1$; unequal symbols give $H[i,j] = 0$, because a contiguous match cannot cross a mismatch. The answer is $\max_{i,j} H[i,j]$, not necessarily the bottom-right entry. Here $H[4,3] = 3$ whereas $H[4,4] = 0$.

A05-S08

Every nonempty alignment ends in symbol/symbol, symbol/gap, or gap/symbol. Removing that column leaves a legal alignment for $(i-1,j-1)$, $(i-1,j)$, or $(i,j-1)$. Its prefix score is bounded above by the corresponding F value. Adding the last-column score yields the upper bound given by the largest candidate. Conversely, append each listed column to an optimal predecessor alignment to attain its candidate. Thus the largest candidate is also achievable. Empty boundaries contain only symbol/gap columns and score $-3i$ or $-3j$.

Full-table computation takes $\Theta(mn)$ time and space. With values or choices retained, traceback takes $O(m+n)$ time; the output itself has length $O(m+n)$. Score-only computation uses two rows, still $\Theta(mn)$ time but $O(\min(m,n))$ auxiliary space, without ordinary full traceback history.

The three candidates classify legal final columns. Equal symbols do not make either gap form illegal. Keeping all three candidates preserves the proof. Removing a candidate would need a separate dominance proof for the specified scoring rule; the LCS recurrence concerns a different objective and supplies no such proof.

A05-I01 Complete matrix-chain calculation

All diagonal entries are 0. Length 2: $m_{12} = 150, s = 1$; $m_{23} = 360, s = 2$; $m_{34} = 180, s = 3$. Length 3: $[1, 3]$ candidates are $0 + 360 + 600 = 960$, $150 + 0 + 180 = 330$, choosing 2. For $[2, 4]$: $0 + 180 + 150 = 330$, $360 + 0 + 600 = 960$, choosing 2. Length 4: $0 + 330 + 250 = 580$, $150 + 180 + 75 = 405$, $330 + 0 + 300 = 630$, choosing 2.

Reconstruct $((A_1A_2)(A_3A_4))$: left output 5×3, right 3×5, root 5×5. Cost $150 + 180 + 75 = 405$. Every dependency is a shorter interval, validating the order.

A05-I02 The cheapest merge alone is insufficient

Split 1 has merge $3 \cdot 2 \cdot 100 = 600$, right-child cost $2 \cdot 4 \cdot 100 = 800$, total 1400. Split 2 has merge $3 \cdot 4 \cdot 100 = 1200$, left-child cost $3 \cdot 2 \cdot 4 = 24$, total 1224. The cheapest-merge rule chooses split 1 and loses. Child costs can outweigh a merge advantage. The optimum is $((A_1A_2)A_3)$, with intermediate 3×4 and final 3×100.

A05-I03 Proof and exact candidate counts

Every parenthesization of $[i, k]$ outputs $p_{i-1} \times p_k$. Replacement changes only internal cost; compatibility and merge cost $p_{i-1}p_kp_j$ remain fixed. Length L has $n - L + 1$ intervals with $L - 1$ splits each. Set $t = L - 1$: $\sum_{t=1}^{n-1}(n - t)t = n\frac{n(n-1)}{2} - \frac{(n-1)n(2n-1)}{6} = (n^3 - n)/6$.

For $n = 4$, the count is 10: 3 at length 2, 4 at length 3, 3 at length 4. Five counts complete parenthesizations; ten counts local candidates across all subproblems. DP reuses child optima rather than enumerating all complete trees.

A05-I04 Full OBST example B

Empty costs are .10, .05, .15, .10. Singleton masses are .35, .50, .35; candidates are $.10 + .05 + .35 = .50$, $.05 + .15 + .50 = .70$, $.15 + .10 + .35 = .60$.

$[1, 2]$ has mass .80: root 1 gives $.10 + .70 + .80 = 1.60$; root 2 gives $.50 + .15 + .80 = 1.45$. $[2, 3]$ has mass .70: root 2 gives $.05 + .60 + .70 = 1.35$; root 3 gives $.70 + .10 + .70 = 1.50$. $[1, 3]$ has mass 1: root 1 gives $.10 + 1.35 + 1 = 2.45$; root 2 gives $.50 + .60 + 1 = 2.10$; root 3 gives $1.45 + .10 + 1 = 2.55$.

The optimum has root 2, children 1 and 3. Success contributes .90; all external depths are 2. Hence $C = .90 + .40 \cdot 2 = 1.70$, $E = .90 + .40 \cdot 3 = 2.10$.

A05-I05 Diagnose mixed empty-cost conventions

Both may be correct if they label their objectives. The real root has depth 0 and both external leaves depth 1. $C = .6 \cdot 1 + (.1 + .3) \cdot 1 = 1$; $E = .6 \cdot 1 + (.1 + .3) \cdot 2 = 1.4$. Each unsuccessful event adds exactly one extra check under E, so $E - C = \sum q$. Zero empty costs while claiming E, or q empty costs while claiming real-key comparisons, mixes conventions. The shape-independent constant preserves optimal trees.

A05-I06 Global mass and conditional expectation

$.50/.35$ is conditional on reaching the left interval, whereas .60 is a global weighted right cost: the denominators differ. Correctly, $E[1,3] = .35 \cdot (.50/.35) + .35 \cdot (.60/.35) + 1 = 2.10$. Both child masses are .35, with root success mass .30, totaling 1.

Multiplying all p, q by 20 scales every w, C, E by 20. Root ordering and ties are unchanged. Global costs become $E = 42, C = 34$; dividing by total weight 20 recovers expectations 2.10 and 1.70. Locally normalized values cannot be inserted directly into the original recurrence.

A05-I07 Propagate ties through reconstruction

$w_{34} = 1 + 1 + 1 + 1 + 1 = 5$, and roots 3 and 4 both give $5 + 3 = 8$. With $w_{14} = 16$, root candidates are $16 + 19 = 35$, $16 + 8 + 8 = 32$, $16 + 19 + 3 = 38$, $16 + 25 = 41$. The global root is uniquely 2, but its right child can be root 3 with right child 4, or root 4 with left child 3. The left child is fixed at 1: two optima.

Since $\sum q = 8$, $E = 32 + 8 = 40$. Total weight 16 gives expected real comparisons $32/16 = 2$ and extended cost $40/16 = 2.5$. A unique global root does not imply a unique optimal tree.

A05-I08 Exact OBST candidate and output bounds

Length L has $n - L + 1$ intervals with L roots each: $\sum_{L=1}^{n}(n - L + 1)L = (n + 1)\sum L - \sum L^2 = n(n + 1)(n + 2)/6$. For $n = 3$, the result is 10: 3 singleton candidates, 4 pair candidates, 3 full-interval candidates.

There are $O(n^2)$ cells and at most n roots per cell, so tied-root lists use at most $O(n^3)$ space. If all nonnegative weights are zero, every BST is optimal; the Catalan count $\frac{1}{n+1}\binom{2n}{n}$ grows exponentially. A compressed decision table and explicitly listing all solutions have different output costs. One tree still takes $\Theta(n)$ to reconstruct.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 15, Dynamic Programming. MIT 6.046J, Fall 2004, Lecture 12, Dynamic Programming. Lecturer Charles E. Leiserson. MIT 6.046J, Fall 2005, Lecture 15, Dynamic Programming. Lecturer Charles E. Leiserson; slides credited to Erik D. Demaine and Charles E. Leiserson. Original scanned algorithms notes (演算法_wjungle), dynamic programming pages; original scanned data structures notes (資料結構), physical pages 166–170, for comparison of optimal search tree models. These notes cover algorithms and data structures and are not the full text of a single textbook.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study